

# Documento Tecnico — Python Socket Examples (TCP & UDP)

## Struttura consigliata del repository

```
python-socket-examples/  
├── README.md  
├── DOCUMENTAZIONE_TECNICA.md  
├── tcp/  
│   ├── tcp_server.py  
│   ├── tcp_client.py  
│   ├── tcp_server_multiclient.py  
│   └── tcp_server_threaded.py  
├── udp/  
│   ├── udp_server.py  
│   ├── udp_client.py  
│   ├── udp_server_counter.py  
│   ├── udp_client_counter.py  
│   ├── udp_server_loss.py  
│   └── udp_client_loss.py  
├── calculator/  
│   ├── calculator_server.py  
│   └── calculator_client.py  
└── diagrams/  
    ├── tcp_flow.png  
    ├── udp_flow.png  
    └── tcp_vs_udp.png
```

## DOCUMENTAZIONE TECNICA

## Python Socket Examples — TCP & UDP

### Introduzione

Questo progetto ha l'obiettivo di mostrare il funzionamento delle socket di rete in Python utilizzando i protocolli TCP e UDP.

Gli esercizi sviluppati permettono di comprendere:

- la comunicazione client/server;
- le differenze tra TCP e UDP;
- il concetto di handshake;
- la gestione delle connessioni;
- la gestione dei timeout;
- la simulazione della perdita di pacchetti;
- la gestione di client multipli;
- la progettazione di un protocollo applicativo personalizzato.

Tutto il codice è stato sviluppato esclusivamente con la libreria standard di Python (`socket`, `threading`, `time`, `random`).

## Exercise 0 — Refactoring e ottimizzazione del codice

### Obiettivo

L'obiettivo dell'esercizio consisteva nel riscrivere il codice originale migliorandone:

- leggibilità;
- modularità;
- riusabilità;
- manutenzione;
- gestione degli errori.

## Modifiche apportate

### 1. Introduzione delle funzioni

#### Problema del codice originale

Nel codice iniziale tutta la logica era concentrata nel blocco principale (`main`), rendendo difficile:

- riutilizzare il codice;
- effettuare debugging;
- aggiungere nuove funzionalità.

## **Soluzione adottata**

Il codice è stato suddiviso in funzioni specializzate.

Esempio:

```
def create_server_socket():
```

```
def handle_client():
```

```
def send_ping():
```

## **2. Eliminazione dei “magic numbers”**

### **Problema**

Nel codice originale erano presenti valori numerici scritti direttamente nel codice:

```
1024
5000
2
```

Questo riduceva la leggibilità.

### **Soluzione**

Sono state introdotte costanti globali:

```
HOST = "127.0.0.1"
PORT = 5000
BUFFER_SIZE = 1024
TIMEOUT = 2
```

## **3. Miglioramento della gestione delle eccezioni**

## UDP timeout

Nel protocollo UDP non esiste garanzia di consegna.

Per evitare blocchi infiniti:

```
client_socket.settimeout(2)
```

e:

```
except socket.timeout:
```

## 4. Miglioramento della leggibilità

Sono stati aggiunti:

- commenti dettagliati;
- docstring;
- messaggi di log più chiari.

Esempio:

```
print(f"[UDP Server] Received '{message}' from {address}")
```

## 5. Chiusura corretta delle socket

Le socket vengono sempre chiuse correttamente:

```
client_socket.close()  
server_socket.close()
```

Questo evita:

- memory leak;
- porte bloccate;
- connessioni zombie.

# Exercise 1 — UDP Message Counter

## Obiettivo

Estendere il server UDP in modo che tenga traccia del numero totale di messaggi **PING** ricevuti.

## Modifiche apportate

### Introduzione del contatore

È stata aggiunta la variabile:

```
ping_count = 0
```

Ogni volta che il server riceve un **PING**:

```
ping_count += 1
```

### Modifica del messaggio di risposta

Prima:

PONG

Dopo:

PONG #3

Implementazione:

```
reply = f"PONG #{ping_count}"
```

### Modifica del client

Il client è stato aggiornato per stampare il messaggio completo:

```
print(data.decode())
```

# Considerazioni tecniche

Il contatore è salvato solamente nella memoria RAM del server.

Se il server viene riavviato:

- il contatore torna a zero;
- non esiste persistenza dei dati.

Per mantenere il contatore dopo il riavvio sarebbe necessario:

- utilizzare un file;
- oppure un database.

## Exercise 2 — TCP Multi-Client Server

### Parte 1 — Server sequenziale

#### Obiettivo

Permettere al server TCP di gestire più client consecutivamente.

### Modifiche apportate

#### Ciclo **accept()**

Nel codice originale:

`accept()`

veniva eseguito una sola volta.

Ora:

```
while clients_served < MAX_CLIENTS:
```

## Introduzione di un limite massimo

È stata introdotta la costante:

```
MAX_CLIENTS = 3
```

Questo permette al server di terminare in modo controllato.

## Funzionamento

1. Il server accetta un client.
2. Comunica con il client.
3. Il client si disconnette.
4. Il server torna in ascolto.

## Parte 2 — Versione multi-thread

### Obiettivo

Gestire più client contemporaneamente.

## Modifiche apportate

### Utilizzo del modulo threading

Import:

```
import threading
```

## Creazione di un thread per ogni client

```
thread = threading.Thread(  
    target=handle_client,  
    args=(client_socket, address)  
)
```

## Vantaggi

Con questa soluzione:

- più client possono comunicare contemporaneamente;
- il server non rimane bloccato su un singolo client;
- aumenta la scalabilità.

## Svantaggi

- maggiore utilizzo di memoria;
- necessità futura di sincronizzazione;
- maggiore complessità.

## Exercise 3 — Simulazione di perdita pacchetti UDP

### Obiettivo

Simulare una rete instabile.

## Modifiche apportate

### Introduzione della probabilità di perdita

DROP\_PROBABILITY = 0.3

## Utilizzo del modulo random

```
import random
```

## Simulazione della perdita

```
if random.random() < DROP_PROBABILITY:
```

In questo caso il server NON invia il pacchetto.

## Effetti osservati

Il client:

- a volte riceve **PONG**;
- a volte riceve timeout.

## Importanza del timeout

Senza timeout:

```
recvfrom()
```

rimarrebbe bloccato indefinitamente.

Con:

```
settimeout(2)
```

il programma continua a funzionare correttamente.

# Exercise 4 — Protocollo personalizzato

## Calculator TCP Application

### Obiettivo

Realizzare un protocollo client/server personalizzato.

## Motivazione della scelta di TCP

TCP garantisce:

- ordine dei messaggi;
- affidabilità;
- assenza di perdita dati.

Per un calcolatore queste proprietà sono fondamentali.

## Protocollo applicativo

### Formato dei messaggi

ADD 5 3  
SUB 8 2  
MUL 4 7  
DIV 10 2

## Gestione errori

Se il client invia:

ABC 5 3

oppure:

ADD hello world

il server risponde:

ERROR

## Terminazione della comunicazione

Il client invia:

QUIT

Il server chiude la connessione.

## Architettura dell'applicazione

### Server

Il server:

1. accetta la connessione;
2. riceve il comando;
3. esegue l'operazione;
4. restituisce il risultato.

### Client

Il client:

1. legge input da tastiera;
2. invia il comando;
3. riceve il risultato;
4. stampa il risultato.

# Considerazioni finali

Questo progetto ha permesso di approfondire:

- socket TCP;
- socket UDP;
- gestione dei timeout;
- gestione client multipli;
- threading;
- progettazione di protocolli;
- gestione errori;
- simulazione di reti instabili.

## Differenze principali tra TCP e UDP

| TCP                        | UDP              |
|----------------------------|------------------|
| Orientato alla connessione | Connectionless   |
| Affidabile                 | Non affidabile   |
| Ordinato                   | Non ordinato     |
| Più lento                  | Più veloce       |
| Handshake                  | Nessun handshake |
| Controllo errori avanzato  | Minimo overhead  |

## Conclusioni

Gli esercizi sviluppati mostrano concretamente:

- come funziona una comunicazione client/server;
- le differenze architetturali tra TCP e UDP;
- l'importanza della gestione degli errori;
- il funzionamento delle socket a basso livello.

Il progetto rappresenta una base solida per sviluppi futuri più avanzati come:

- chat multiutente;

- file transfer;
- web server;
- protocolli custom;
- sistemi distribuiti.